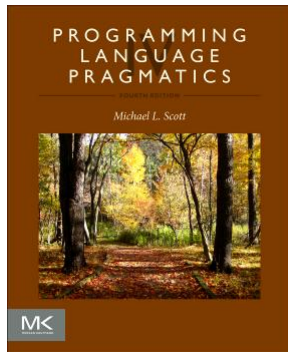


Chapter 7:: Data Types

Programming Language Pragmatics, Fourth Edition

Michael L. Scott



Data Types

- **Most programming languages include a notion of *type*** for expressions and/or objects. Types serve two principal purposes:
- implicit context or many operations (so that the programmer does not have to specify that context explicitly)
 - Examples
 - In C, for instance, the expression $a + b$ will use integer addition if a and b are of integer type
 - the operation `new p` in Pascal, where p is a pointer, will allocate a block of storage from the heap that is the right size to hold an object of the type pointed to by p
 - In C++, Java, and C#, the operation `new my_type()` not only allocates (and returns a pointer to) a block of storage sized for an object of type `my_type` and it also automatically calls any user-defined initialization (constructor) function that has been associated with that type
- checking - make sure that certain meaningless operations do not occur
 - They prevent the programmer from adding a character and a record, for example, or passing a file as a parameter to a subroutine that expects an integer.
 - type checking cannot prevent all meaningless operations

Data Types (Type Systems)

- Computer hardware can interpret bits in memory in several different ways:
 - instructions
 - addresses
 - characters
 - integers, floats, etc.
- bits themselves are untyped eg: Assembly language
- high-level languages associate types with value, useful for operator context and allows error checking

type system

a type system consists of

- a way to define types and associate them with language constructs

- constructs that must have values include constants, variables, record fields, parameters, literal constants, subroutines, and complex expressions

- Rules

- type equivalence: define when the types of two values are the same.

- type compatibility: define when a value of a given type can be used in a particular context.

- type inference: define type of an expression based on the types of its parts, ex. polymorphism.

type checking

Type checking is the process of ensuring that a program obeys the language's type compatibility rules. A violation of the rules is known as a *type clash*.

- a language in which the compiler or the run-time system can guarantee that the programs it accepts will execute without type errors called strongly typed.
- a language is statically typed if it is strongly typed and type checking can be performed at compile time
 - few languages fall completely in this category

// Java example

```
int num;  
num = 5;
```

// Groovy example

```
num = 5
```

type checking

- Examples
 - Ada is statically typed
 - Java is strongly typed, with a non-trivial mix of things that can be checked statically and things that have to be checked dynamically
 - C is strongly typed

type checking

- A language is dynamically typed if type checking is performed at run time
 - late binding
 - List and Smalltalk
 - scripting languages, such as Python and Ruby

The Meaning of “Type”

we all have developed an intuitive notion of what types are; what's behind the intuition?

There are at least three ways to think about types:

- denotational approach: From the denotational point of view, a type is simply a set of values. A value if belong to this set has same type.
- constructive approach: From the constructive point of view, a type is either collection of *built-in* types (integer, character, Boolean, real, etc.; also called *primitive* or *predefined* types), or a *composite* type eg: Record, array, set.
- abstraction-based approach: From the abstraction-based point of view, a type is an *interface* define operations and semantics.
- For most programmers types usually mixture of these viewpoints.

Data Types

specified in different ways

- Fortran: spelling of variable names
- Lisp: infer types at compile time
- List, Smalltalk: track at run time
- C, C++: declared at compile time

ORTHOGONALITY

- is a useful goal in the design of a language to make the various features of the language as *orthogonal* as possible.
- Orthogonality means that features can be used in any combination, the combinations all make sense, and the meaning of a given feature is *consistent*, regardless of the other features with which it is combined.

Type Systems

- It is also important in type system
 - A collection of features is orthogonal if there are no restrictions on the ways in which the features can be combined.
 - Makes the language easy to learn, easy to understand, and easy to use
 - A relatively small set of primitive constructs can be combined in a relatively small number of ways to build the control and data structures of the language.
 - Every possible combination is legal and meaningful.
 - The more orthogonal the design of a language, the fewer exceptions the language rules require.

Type Systems

- ORTHOGONALITY

-Example: Adding two 32-bit integer values that reside in either memory or registers and replacing one of two values with the sum.

- The **IBM mainframes** have two instructions:
- A Reg1, memory_cell
 //Reg1 <- contents (Reg1) + contents(memory_cell)
- AR Reg1, Reg2
 //Reg1 <- contents (Reg1) + contents(Reg2)
 where Reg1 and Reg2 represent registers.

Type Systems

- Example:
 - The type systems of C and Pascal are more orthogonal than that of Fortran. The array elements in traditional Fortran were always of scalar type, C and Pascal allow arbitrary types.
- Non-orthogonality
- Pascal requires the bounds of each array to be specified at compile time
- C requires a lower bound of zero on all array indices

Type Systems

- Common terms:
 - discrete types – countable
 - integer
 - boolean
 - char
 - enumeration
 - subrange
 - scalar types (simpler types) - one-dimensional
 - discrete
 - real

Type Systems

- Composite types:
 - records (unions)
 - arrays
 - strings
 - sets
 - pointers
 - lists
 - files

Implementation of Data Types

- Booleans
- Character Type
- Numeric Types
- Real
- Enumeration
- Subtype

Boolean Type

- Booleans are typically implemented as single-byte quantities, with 1 representing true and 0 representing false.
- C was historically unusual in omitting a Boolean type: where most languages would expect a Boolean value, C expected an integer, using zero for false and anything else for true.
- C99 introduced a new `_Bool` type, but it is effectively an integer that the compiler is permitted to store in a single bit.

Character Type

- Characters are implemented as one-byte quantities, typically (but not always) using the ASCII encoding.
- More recent languages (e.g., Java and C#) use a two-byte representation designed to accommodate (the commonly used portion of) the *Unicode* character set.
- Unicode (preferred character encoding) is an international standard designed to capture the characters of a wide variety of languages.
- Formats:
 - UTF-8 encoding: Variable-width encoding (1, 2, 3 or 4 bytes)
 - UTF-16 encoding: Variable-width encoding (2, or 4 bytes)
 - UTF-32 encoding: Fixed-width encoding (4 bytes)
- The first 128 characters of Unicode (\u0000 through \u007f) are identical to ASCII.

Numeric Types

- A few languages (e.g., C and Fortran) distinguish between different lengths of integers and real numbers; most do not, and leave the choice of precision to the implementation.
- Differences in precision across language implementations lead to a lack of portability: programs that run correctly on one system may produce run-time errors or erroneous results on another.
- Java and C# are unusual in providing several lengths of numeric types, with a specified precision for each.

Integer Representation

- Unsigned integers represented with N bit binary can represent 2^N (positive) numbers
- To handle positive and negative numbers, the sign is an extra piece of information that must be encoded.
- Two's Complement is the most common representation

•Two's Complement

- MSB becomes the SIGN bit (1 indicates a negative number)
- Can now represent signed integers -2^{N-1} to $+2^{N-1} - 1$ (e.g., -128 to 127)
- Algorithm #1: complement binary number and then add 1
- Algorithm #2: start with LSB, copy bits up to and including the first 1, then invert all remaining bits

Example: 14 = %0000 1110
-14 = %1111 0010

Important Note: The computer generally does not know that a bit sequence represents a signed number. It is the programmer's (i.e., your) responsibility!

Adding Precision (two's complement)

What if we want to take a two's complement number and add bits to it?

Take whatever the Sign bit is and extend it to the left.

$$\begin{aligned}(127)_{10} &= \%01111111 \text{ (8 bits)} \\ &= \%0000000000111111 \text{ (16 bits)} \\ &= \%00\dots\dots\dots1111111 \text{ (32 bits)}\end{aligned}$$

$$\begin{aligned}(-128)_{10} &= \%10000000 \text{ (8 bits)} \\ &= \%1111111110000000 \text{ (16 bits)} \\ &= \%11\dots\dots\dots10000000 \text{ (32 bits)}\end{aligned}$$

This is called Sign Extension.

Little-Endian vs. Big-Endian

- Numbering the bits from right to left, beginning with zero is called Little Endian byte order. Or Higher address store lower byte and vice versa. Intel 80x86 and DECstation 3100 use the Little Endian byte ordering.
- Numbering the bits from left to right, beginning with zero is called Big Endian byte order. SunSparc and Macintosh use the Big-Endian byte ordering.

Little-Endian vs. Big-Endian

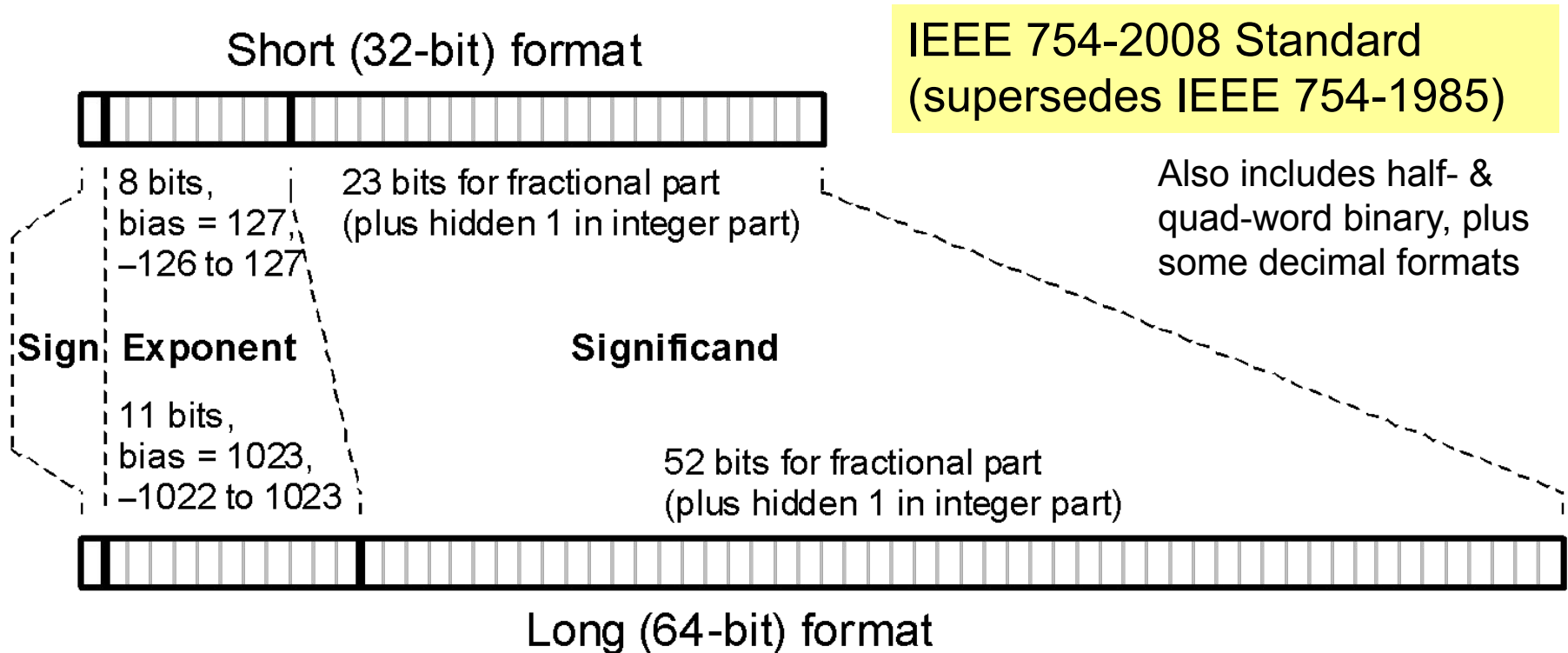
Big Endian

12	34	56	78
<i>0x00400000</i>	<i>0x00400001</i>	<i>0x00400002</i>	<i>0x00400003</i>

Little Endian

78	56	34	12
<i>0x00400000</i>	<i>0x00400001</i>	<i>0x00400002</i>	<i>0x00400003</i>

The IEEE Floating-Point Standard



The IEEE standard floating-point number representation formats.

Enumerated Type

- Enumerations facilitate the creation of readable programs, and allow the compiler to catch certain kinds of programming errors.
- An enumeration type consists of a set of named elements. Comparisons are generally valid since ordered, and there is usually a mechanism to determine the predecessor or successor of an enumeration value
- **Pascal type definition:**
 - `type weekday = (sun, mon, tue, wed, thu, fri, sat);`
Valid operations: `mon < tue`
`tomorrow = Succ(today)`
- **C type definition:**
 - `enum weekday {sun, mon, tue, wed, thu, fri, sat};`
is essentially equivalent to
`typedef int weekday; const weekday sun = 0, mon = 1, tue = 2, wed = 3, thu = 4, fri = 5, sat = 6;`

Enumerated Type Cont..

- In C and C++ languages an enumeration type is actually an integer type, and each enumerator denotes a small integer starting at zero.
- **C++ type definition:**
 - `enum Month {jan, feb, mar, apr, may, jun, jul, aug, sep, oct, nov, dec};`
 - defines Month to be an integer type, and binds jan to 0, feb to 1, and so on. Thus:
 - `Month = {0, 1, 2, . . . , 11}`

Subrange Type

- A subrange is a type whose values compose a contiguous subset of the values of some discrete *base* type (also called the *parent* type)

- In Pascal and most of its descendants, one can declare subranges of integers, characters e.t.c:

- **Pascal,**

```
type test_score = 0..100;  
workday = mon..fri;
```

- **Ada:**

```
type test_range is new integer range from 0 to 100  
subtype workday is weekday range from mon..fri
```

Subrange Type Cont..

- The **range...** portion of the definition is called a type constraint.
- **test_score** is a derived type, incompatible with integers.
- The **workday** type, on the other hand, is a constrained subtype.

Storage requirements for Subrange Type

□ **Compiler's Viewpoint**

- Compiler analyzes a subrange declaration, it knows the expected range of subrange values, and can generate code to perform dynamic semantic checks to ensure that no subrange variable is ever assigned an invalid value.
- Also, since the compiler knows the number of values in the subrange, it can sometimes use fewer bits to represent subrange values than it would need to use to represent arbitrary integers.
- In the example provided, **test_score** values can be stored in a single byte.

Storage requirements for Subrange Type

□ **Example:**

- type water_temperature = 273..373; (* degrees Kelvin *) would be stored in at least **two** bytes.
- 101 distinct values in the type, the largest (373) is too large to fit in a single byte in its natural encoding.
- An unsigned byte can hold values in the range 0 . . 255; a signed byte can hold values in the range -128 . . 127.

Type Checking

- In most statically typed languages, every definition of an object (constant, variable, subroutine, etc.) must specify the object's type. Moreover, many of the contexts in which an object might appear are also typed so that valid processing can be done.
- A TYPE SYSTEM has rules for
 - type equivalence (when are the types of two values the same?)
 - type compatibility (when can a value of type A be used in a context that expects type B?)
 - type inference (what is the type of an expression, given the types of the operands?)
- Of the three, type compatibility is the one of most concern to programmers. It determines when an object of a certain type can be used in a certain context

Type Checking

- Type compatibility / type equivalence
 - Compatibility is the more useful concept, because it tells you what you can DO
 - The terms are often (incorrectly, but we do it too) used interchangeably.

type equivalence

- Certainly format does not matter:

```
struct { int a, b; }
```

is the same as

```
struct {  
    int a, b;  
}
```

and we certainly want them to be the same as

```
struct {  
    int a;  
    int b;  
}
```

type equivalence

- In a language in which the user can define new types, there are two principal ways of defining type equivalence: structural equivalence and name equivalence
 - *Name equivalence* is based on the *lexical occurrence* of type definitions: roughly speaking, each definition introduces a new type.
 - *Structural equivalence* is based on the *content* of type definitions: roughly speaking, two types are the same if they consist of the same components

type equivalence

- To determine if two types are structurally equivalent, a compiler can expand their definitions by replacing any embedded type names with their respective definitions, recursively, until nothing is left but a long string of type constructors, field names, and built-in types. If these expanded strings are the same, then the types are equivalent.

```
1. type student = record
2.     name, address : string
3.     age : integer

4. type school = record
5.     name, address : string
6.     age : integer

7. x : student;
8. y : school;
9. ...
10. x := y;           -- is this an error?
```

if programmers accidentally assigned a value of type `school` into a variable of type `student`, the structural equivalence compiler will accept such an assignment.

Name equivalence is based on the assumption that if the programmer goes to the effort of writing two type definitions, then those definitions are probably meant to represent different types.

type equivalence

- Type equivalence is structural equivalence: $T1 \equiv T2$ if and only if $T1$ and $T2$ have the same set of values.
- It may be checked by comparing the structures of the types $T1$ and $T2$. (It is unnecessary, and in general even impossible, to enumerate all values of these types.)
- **Rules:**
 - If $T1$ and $T2$ are both primitive, then $T1 \equiv T2$ if and only if $T1$ and $T2$ are identical.
 - For example: $\text{Integer} \equiv \text{Integer}$ but $\text{Integer} \not\equiv \text{Float}$
(The symbol “ $\not\equiv$ ” means “is not equivalent to”.)

type equivalence

- If $T1 = A1 \times B1$ and $T2 = A2 \times B2$, then $T1 \equiv T2$ if and only if $A1 \equiv A2$ and $B1 \equiv B2$.
- For example:
- $\text{Integer} \times \text{Float} \equiv \text{Integer} \times \text{Float}$ but $\text{Integer} \times \text{Float} \not\equiv \text{Float} \times \text{Integer}$
- If $T1 = A1 \rightarrow B1$ and $T2 = A2 \rightarrow B2$, then $T1 \equiv T2$ if and only if $A1 \equiv A2$ and $B1 \equiv B2$. For example:
- $\text{Integer} \rightarrow \text{Float} \equiv \text{Integer} \rightarrow \text{Float}$ but
- $\text{Integer} \rightarrow \text{Float} \not\equiv \text{Integer} \rightarrow \text{Boolean}$
- If $T1 = A1 + B1$ and $T2 = A2 + B2$, then $T1 \equiv T2$ if and only if either
- $A1 \equiv A2$ and $B1 \equiv B2$, or $A1 \equiv B2$ and $B1 \equiv A2$.
- For example:
- $\text{Integer} + \text{Float} \equiv \text{Integer} + \text{Float}$
- $\text{Integer} + \text{Float} \equiv \text{Float} + \text{Integer}$ but
- $\text{Integer} + \text{Float} \not\equiv \text{Integer} + \text{Boolean}$
- Otherwise, $T1 \not\equiv T2$.

type equivalence

- Another definition of type equivalence is name equivalence: $T1 \equiv T2$ if and only if $T1$ and $T2$ are structurally equivalent without name substitution.

Example 1

- `struct Position { int x, y; };`
- `struct Position pos;`
- `struct Date { int m, d; };`
- `struct Date today;`
- `void show (struct Date d);`
- call “`show(today);`” would pass its type check, whether the language adopts structural or name equivalence.
- call “`show(pos);`” would pass its type check only if the language adopts structural equivalence

type equivalence

- Consider the following declarations (in a different part of the program):
 - `struct OtherDate { int m, d; };`
 - `struct OtherDate tomorrow;`
- Call “`show(tomorrow);`” would pass its type check only if the language adopts structural equivalence.
- It would fail its type check if the language adopts name equivalence, since the type of `tomorrow` is not name-equivalent to the type of the parameter of `show`: the two types were defined in different places.

type equivalence

- Name equivalence forces each distinct type to be defined in one and only with one name. This is sometimes inconvenient, but it helps to make the program more maintainable. (If the same type is defined in several places, and subsequently it has to be changed, the change must be made consistently in several places.)
- Structural equivalence allows confusion between types that are only coincidentally similar such as Position and Date in **Example 1**
- Structural equivalence is adopted by ALGOL68.
- Name equivalence is adopted by most programming languages, including C, C++ and ADA.

type equivalence

- There are at least two common variants on name equivalence
 - A language in which aliased types are considered distinct is said to have *strict name equivalence*.
 - A language in which aliased types are considered equivalent is said to have *loose name equivalence*.

```
1. type cell    = ...           -- whatever
2. type alink   = pointer to cell
3. type blink   = alink
4. p, q : pointer to cell
5. r    : alink
6. s    : blink
7. t    : pointer to cell
8. u    : alink
```

Here the declaration at line 3 is an alias; it defines `blink` to be “the same as” `alink`.

Under strict name equivalence, line 3 is both a declaration and a definition, and `blink` is a new type, distinct from `alink`.

Under loose name equivalence, line 3 is just a declaration; it uses the definition at line 2.

Under strict name equivalence, `p` and `q` have the same type, because they both use the *anonymous* type definition on the right-hand side of line 4, and `r` and `u` have the same type, because they both use the definition at line 2.

Under loose name equivalence, `r`, `s`, and `u` all have the same type, as do `p` and `q`.

Type conversion

- An explicit request by the programmer for the compiler to insert code into the program to convert one type to another, compatible type.
 - `int x; float y = 6.3; x = (int)y;`
- There are three principal cases of type conversion:
 1. The types would be considered structurally equivalent. In this case the types employ the same low-level representation, and have the same set of values. The conversion is therefore a purely conceptual operation; no code will need to be executed at run time.
 2. The types have different sets of values, but the intersecting values are represented in the same way. Eg: one is two's complement signed integer, other is unsigned integer.
 3. The types have different low-level representations, but we can nonetheless define some sort of correspondence among their values. A 32-bit integer, for example, can be converted to a double-precision IEEE floating-point number with no loss of precision.
- A change of type that does not alter the underlying bits is called a *nonconverting type cast*, or sometimes a *type pun*. It should not be confused with use of the term *cast*.

Type conversion

- type **celsius_temp** is new integer;
- **n** : integer; -- assume 32 bits
- **r** : real; -- assume IEEE double-precision
- **t** : test_score; -- type **test_score**: 0..100
- **c** : celsius_temp; -- as above
- **t** := test_score(**n**); -- run-time semantic check required
- **n** := integer(**t**); -- no check req.; every test_score is an int
- **r** := real(**n**); -- requires run-time conversion /* change Lower Rep*/
- **n** := integer(**r**); -- requires run-time conversion and check
- **n** := integer(**c**); -- no run-time code required
- **c** := celsius_temp(**n**); -- no run-time code required

In C language:

```
r = (float) n; /* generates code for run-time conversion */
```

```
n = (int) r; /* also run-time conversion, with no overflow check */
```

Type Compatibility

- Most languages do not require equivalence of types in every context. Instead, they merely say that a value's type must be *compatible* with that of the context in which it appears. In an assignment statement, the type of the right-hand side must be compatible with that of the left-hand side. The types of the operands of $+$ must both be compatible with some common type that supports addition (integers, real numbers, or perhaps strings or sets).
- In a subroutine call, the types of any arguments passed into the subroutine must be compatible with the types of the corresponding formal parameters.

Type Compatibility

- Coercion

- Whenever a language allows a value of one type to be used in a context that expects another, the language implementation must perform an automatic, implicit conversion to the expected type. This conversion is called a *type coercion*.
- The compiler performs an implicit conversion from one type to another, compatible type without informing the user. This conversion could cause a logic error at run-time if it is not dynamically checked. For example:
 - `int x; short y; y = x;`

The compiler does an implicit coercion by taking the low-order 15 bits of `x` and assigning them to `y`. It also preserves the sign bit. This works well if the value of `x` fits in 15 bits and poorly otherwise.

Type Compatibility

- C has lots of coercion, but with simpler rules:
 - all **floats** in expressions become **doubles**
 - **short**, **int**, and **char** become **int** in expressions
 - if necessary, precision is removed when assigning into LHS

Type coercion

short int s;

unsigned long int l;

char c; /* may be signed or unsigned – implementation dependent*/

float f; /* usually IEEE single-precision */

double d; /* usually IEEE double-precision */

Suppose that these variables are 16, 32, 8, 32, and 64 bits in length

s = l;

Type coercion

short int s;

unsigned long int l;

char c; /* may be signed or unsigned -
implementation-dependent*/

float f; /* usually IEEE single-precision */

double d; /* usually IEEE double-precision */

Suppose that these variables are 16, 32, 8, 32, and 64 bits in
length

s = l; /* l's low-order bits are interpreted as a signed number. */

l = s;

Type coercion

short int s;

unsigned long int l;

char c; /* may be signed or unsigned -
implementation-dependent*/

float f; /* usually IEEE single-precision */

double d; /* usually IEEE double-precision */

Suppose that these variables are 16, 32, 8, 32, and 64 bits in
length

s = l; /* l's low-order bits are interpreted as a signed number. */

l = s; /* s is sign-extended to the longer length, then its bits are
interpreted as an unsigned number. */

s = c;

Type coercion

short int s;

unsigned long int l;

char c; /* may be signed or unsigned -
implementation-dependent*/

float f; /* usually IEEE single-precision */

double d; /* usually IEEE double-precision */

Suppose that these variables are 16, 32, 8, 32, and 64 bits in length

s = l; /* l's low-order bits are interpreted as a signed number. */

l = s; /* s is sign-extended to the longer length, then its bits are
interpreted as an unsigned number. */

s = c; /* c is either sign-extended to s's length; the result is then
interpreted as a signed number. */

f = l;

Type coercion

short int s;

unsigned long int l;

char c; /* may be signed or unsigned -
implementation-dependent*/

float f; /* usually IEEE single-precision */

double d; /* usually IEEE double-precision */

Suppose that these variables are 16, 32, 8, 32, and 64 bits in length

s = l; /* l's low-order bits are interpreted as a signed number. */

l = s; /* s is sign-extended to the longer length, then its bits are
interpreted as an unsigned number. */

s = c; /* c is either sign-extended to s's length; the result is then
interpreted as a signed number. */

f = l; /* l is converted to floating-point. Since f has fewer
significant bits, some precision may be lost. */

d = f;

Type coercion

short int s;

unsigned long int l;

char c; /* may be signed or unsigned -
implementation-dependent*/

float f; /* usually IEEE single-precision */

double d; /* usually IEEE double-precision */

Suppose that these variables are 16, 32, 8, 32, and 64 bits in length

s = l; /* l's low-order bits are interpreted as a signed number. */

l = s; /* s is sign-extended to the longer length, then its bits are
interpreted as an unsigned number. */

s = c; /* c is either sign-extended to s's length; the result is then
interpreted as a signed number. */

f = l; /* l is converted to floating-point. Since f has fewer
significant bits, some precision may be lost. */

d = f; /* f is converted to the longer format; no precision is lost.
*/

f = d;

Type coercion

short int s;

unsigned long int l;

char c; /* may be signed or unsigned - implementation-dependent */

float f; /* usually IEEE single-precision */

double d; /* usually IEEE double-precision */

Suppose that these variables are 16, 32, 8, 32, and 64 bits in length

s = l; /* l's low-order bits are interpreted as a signed number. */

l = s; /* s is sign-extended to the longer length, then its bits are
interpreted as an unsigned number. */

s = c; /* c is either sign-extended to s's length; the result is then
interpreted as a signed number. */

f = l; /* l is converted to floating-point. Since f has fewer significant
bits, some precision may be lost. */

d = f; /* f is converted to the longer format; no precision is lost. */

f = d; /* d is converted to the shorter format; precision may be lost. If
d's value cannot be represented in single-precision, the
result is undefined, but NOT a dynamic semantic error. */

Type Inference

- Make sure you understand the difference between
 - type conversions (explicit)
 - type coercions (implicit)
 - sometimes the word 'cast' is used for conversions (C is guilty here)
 - Non converting type casts – interpret bits of one type as if they are of another type

```
r = *((float *) &n);
```

Universal ReferenceTypes

- Systems programming or to facilitate the writing of general-purpose container (collection) objects (lists, stacks, queues, sets, etc.) that hold references to other objects, several languages provide a universal reference type.
- **C and C++:** void *
- **Clu:**any
- **Modula-2:** address, **Modula-3:**refany
- **Java:**Object; **C#:** object.
- Type of the object referred to by a universal reference is unknown, the compiler will not allow any operations to be performed on that object.

Universal Data Type

- If we assign a universal reference to a floating-point number into a variable that is supposed to hold a reference to an integer, then subsequent operations on the “integer” would interpret the bits of the object incorrectly.
- **Implementation:**
 - To ensure the safety of universal to specific assignments is to make objects self-descriptive- to include in the representation of each object a tag that indicates its type
- **Usage:** dynamic method binding in C++
- **Type tags in objects:** takes a nontrivial amount of space, but allow the implementation to prevent the assignment of an object of one type into a variable of another

Universal Data Type

- **Java and C#:**
- A universal to specific assignment requires a **type cast**, and will generate an exception if the universal reference does not refer to an object of the casted type.

```
import java.util.*;      // library containing Stack container class
...
Stack myStack = new Stack();
String s = "Hi, Mom";
Foo f = new Foo();      // f is of user-defined class type Foo
...
myStack.push(s);
myStack.push(f);        // we can push any kind of object on a stack
...
s = (String) myStack.pop();
    // type cast is required, and will generate an exception at run
    // time if element at top-of-stack is not a string
```

Type Inference

- We have seen how type checking ensures that the components of an expression have appropriate types. But what determines the type of the overall expression?
- The result of an arithmetic operator usually has the same type as the operands.
 - The result of a comparison is usually Boolean.
 - The result of a function call has the type declared in the function's header.
 - The result of an assignment has the same type as the left-hand side.
- What about Subranges??
- Given the following Pascal definitions
 - `type Atype = 0..20;`
 - `type Btype = 10..20;`
 - `var a : Atype;`
 - `var b : Btype;`

What is the type of `a + b`?

Neither Atype nor Btype, since the possible values range from 10 to 40.

Type Inference

- A new anonymous subrange type with 10 and 40 as bounds
- In Pascal and its descendants: the result of any arithmetic operation on a subrange has the subrange's base type, in this case **integer**.
- What if the result of an arithmetic operation is assigned into a variable of a subrange type?
 - A dynamic semantic check may be required.
- To avoid the expense of some unnecessary checks, a compiler may keep track at compile time of the largest and smallest possible values of each expression, in essence computing the anonymous 10. . . 40 type.

Type Inference

- C# 3.0 allows a variable declaration to use the placeholder `var` instead of a type name when an appropriate type can be inferred from the initialization expression:
- `var i = 123; // equiv. to int i = 123;`
- `var map = new Dictionary<int, string>(); // equiv. to
//Dictionary<int, string> map = new Dictionary<int, string>();`